

Blazor Software Engineer – Technical Assessment

1. Introduction

This document outlines the technical assessment solution for the Blazor Software Engineer position at Medica. The solution demonstrates the ability to build a Blazor web application that interacts with an API, fetches data from it, and renders the data to the user interface (UI). The purpose of this document is to provide insights into the decisions made, the system's design, and the code's extensibility.

2. Reasoning for Decisions

2.1 Blazor Hosting Model

For the Blazor WebAssembly application, I chose to use the **Blazor WebAssembly (WASM)** hosting model. The rationale for this decision is:

- **Client-side execution:** Blazor WASM runs entirely in the browser, reducing server-side load and making the application more scalable.
- **Better user experience:** As it runs on the client side, there's less need for constant interaction with the server, improving the application's responsiveness.

2.2 API Consumption

I used **HttpClient** to interact with the API. The base URL for the API is set dynamically based on the environment (development or production) to handle different API endpoints. This allows the system to function in both local and deployed environments without requiring code changes.

2.3 Design Patterns & SOLID Principles

I followed SOLID principles while designing the application. For instance:

- **Single Responsibility Principle (SRP):** Each component (service, UI, etc.) has a distinct responsibility.
- **Dependency Injection:** I used DI to inject services like **HttpClient** and the **EmployeeService** into the Blazor components.
- **Interface Segregation:** The **IEmployeeService** interface is used to abstract the employee-related logic from the UI.

- **Open/Closed Principle:** The system is open for extension (by adding new functionalities) but closed for modification, ensuring the application can be extended without changing existing code.
-

3. Solution Architecture

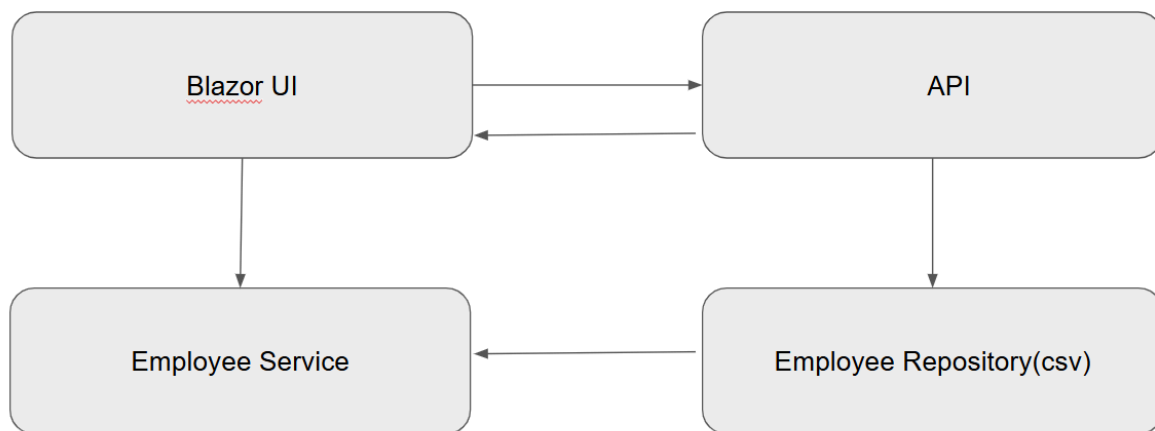
3.1 Overview

The application is composed of:

1. **Blazor WebAssembly Frontend:** Responsible for rendering the UI and interacting with the backend API.
2. **API Backend:** Provides endpoints for employee data management.
3. **Data Storage:** Employee data is stored in a CSV file, which is read, updated, and written by the API.

3.2 Design Diagram

Blazor WebAssembly and API Interaction



4. Features

4.1 Employee CRUD Operations

The application supports the following CRUD operations:

- **Create:** Add new employee details via the `AddEmployee` method.
- **Read:** Retrieve employee data through the `GetEmployeesAsync` method.

- **Update:** Modify existing employee information using the `UpdateEmployee` method.
- **Delete:** Remove an employee entry with the `DeleteEmployee` method.

4.2 Dynamic API Base URL

The API base URL is determined dynamically based on the environment (development or production) using the following condition in `Program.cs`:

```
var baseAddress = builder.HostEnvironment.IsDevelopment()  
    ? "http://localhost:16060" // Development URL  
    : "https://api.productionurl.com"; // Production URL
```

4.3 User Interface

The UI consists of:

- **Employee List:** Displays the list of employees in a user-friendly table.
 - **Forms for Create** Allows the user to add employee records.
 - **Error Handling:** Displays appropriate messages when an operation fails.
-

5. Things that Could be Done Differently

5.1 Use of LocalStorage for Employee Data

- The employee data could be stored in **local storage** or **IndexedDB** on the client-side to improve performance and reduce repeated API calls. This would make the application work offline.

5.2 Unit Tests

- Unit tests could be written for all services, particularly for the `EmployeeService`, to verify that data handling logic works correctly in different scenarios (e.g., when the API is down or returns an error).

5.3 Authentication and Authorization

- In a real-world scenario, the app would likely require **authentication** and **authorization** to protect sensitive employee data. This could be achieved using **JWT tokens** for API calls and implementing **role-based access control** on the Blazor frontend.
-

6. How the Solution Could be Extended or Improved

6.1 API Pagination and Filtering

- **Pagination:** For larger datasets, implementing pagination in the API and displaying paginated results in the UI would enhance performance and user experience.
- **Filtering/Sorting:** Implement search, filtering, and sorting functionalities to allow the user to easily find employees.

6.2 Data Validation and UI Improvements

- **Client-side Validation:** Ensure that the form inputs are validated on the client side before sending them to the server.
- **UI Enhancements:** Use **CSS frameworks** like **Bootstrap** or **Tailwind CSS** for better styling and UI responsiveness.
- Update functionality

6.3 Extending to a Full MVC Application

- The current design could be extended to include a **server-side component** where the API logic is separated from the client, leading to better scalability and maintainability.
-

7. Instructions to Run the Solution

7.1 Prerequisites

- .NET 8.0 or later
- Visual Studio 2022 or later
- Blazer WebAssembly project

7.2 Steps to Run

1. Clone or download the project.
2. Open the solution in **Visual Studio**.
3. Set the project as the startup project.
4. Build and run the solution.